

Relatório do 2º Trabalho prático

UC: Sistemas Distribuídos



**UNIVERSIDADE
DE ÉVORA**

Realizado por:

Diogo Tavares(I58049)

Francisco Rodrigues(I59119)

Introdução

O objetivo deste trabalho é desenvolver um sistema distribuído para monitorizar a temperatura e a humidade de diversas áreas da Universidade de Évora para além da consulta de métricas ambientais divididos por sala, departamento, piso ou edifício, com opções de filtragem.

O sistema está implementado de maneira a permitir também:

- Registrar, consultar, atualizar e eliminar dispositivos IoT através de endpoints RESTful;
- Receber métricas de dispositivos através de três protocolos distintos:
 - MQTT para sensores IoT simples (baixo consumo energético)
 - RPC para dispositivos edge/gateways (alto desempenho)
 - REST para dispositivos com capacidade HTTP (simplicidade)

1. Justificação das Escolhas ao Longo do Trabalho

1.1. Estrutura da Base de Dados (PostgreSQL)

A escolha de uma base de dados relacional (PostgreSQL) deveu-se à nossa familiaridade com este tipo de BD e devido à natureza estruturada dos dados do sistema.

Foram assim definidas duas tabelas principais:

- **Tabela “devices” (Entidade Dispositivo):**
 - Optou-se por separar os dados do dispositivo (Localização: Sala, Piso, Edifício; e Configuração: Protocolo, Estado) das medições.
 - A chave primária é o **id** (String) e permite identificadores semânticos e não só numerais como "sensor-sala-130"
 - Incluiu-se um campo booleano **active**. Isto permite que o histórico dos dados seja preservado mesmo que o sensor seja retirado de serviço, conforme implementado no DeviceController.
- **Tabela “metrics” (Entidade Métrica):**
 - Esta tabela armazena as séries temporais.
 - Utiliza-se a chave estrangeira **device_id** para identificar cada leitura a um só dispositivo.
 - Os tipos de dados double foram escolhidos para Temperatura e Humidade para manter a precisão das leituras, embora os clientes arredondem para 2 casas decimais antes do envio.

1.2. Métodos de Comunicação (Multi-Protocolo)

A arquitetura do servidor foi desenhada para ser agnóstica ao protocolo na camada de negócio (**MetricsProcessor**), mas flexível na camada de ingestão:

1. **REST:** É normalmente utilizado para a gestão administrativa (CRUD de dispositivos) e para sensores com capacidade HTTP pois é o padrão da indústria para integração web por ser fácil de depurar, universalmente compatível e o seu formato JSON garantir legibilidade humana.

2. **gRPC**: Sendo maioritariamente escolhido quando existem gateways ou sensores de alto desempenho por utilizar **Protocol Buffers** (binário) em vez de JSON (texto), o que reduz drasticamente o tamanho da mensagem. Também opera sobre HTTP/2, permitindo multiplexagem e menor latência na conexão, o que o torna ideal para envio frequente de dados.
 3. **MQTT**: Selecionado para os sensores IoT de baixo consumo energético (bateria) devido ao seu modelo Publish/Subscribe, que desacopla o dispositivo do servidor. Por ser assíncrono e extremamente leve (com um overhead mínimo de 2 bytes), torna-se a solução ideal para operar em redes instáveis, permitindo que o sensor envie dados eficientemente sem o desperdício energético de aguardar por respostas síncronas, como ocorre no HTTP.
-

2. Configuração da Base de Dados PostgreSQL

O sistema foi desenhado para ser executado num ambiente contentorizado (Docker), o que automatiza a configuração. No entanto, para efeitos de reprodução manual ou manutenção, têm de se seguir os seguintes passos e permissões.

2.1. Criação da Base de Dados e Utilizador

Aceder à linha de comandos do PostgreSQL (**psql**) e executar:

SQL

```
-- 1. Criar um utilizador dedicado para a aplicação (segurança)  
CREATE USER uevora_admin WITH PASSWORD 'segredo123';
```

```
-- 2. Criar a base de dados  
CREATE DATABASE monitorizacao_db;
```

```
-- 3. Atribuir privilégios  
GRANT ALL PRIVILEGES ON DATABASE monitorizacao_db TO  
uevora_admin;
```

2.2. Esquema das Tabelas (DDL)

Estas são as instruções SQL equivalentes às entidades JPA definidas no projeto (**DeviceEntity** e **MetricEntity**):

SQL

-- Ligar à base de dados

--Tabela de Dispositivos

```
CREATE TABLE devices (  
    id VARCHAR(255) PRIMARY KEY,  
    protocol VARCHAR(50) NOT NULL,  
    room VARCHAR(100),  
    department VARCHAR(100),  
    floor VARCHAR(50),  
    building VARCHAR(100),  
    active BOOLEAN DEFAULT TRUE  
);
```

--Tabela de Métricas

```
CREATE TABLE metrics (  
    id SERIAL PRIMARY KEY,  
    device_id VARCHAR(255) NOT NULL,  
    temperature DOUBLE PRECISION,  
    humidity DOUBLE PRECISION,  
    timestamp TIMESTAMP,  
    CONSTRAINT fk_device FOREIGN KEY (device_id) REFERENCES  
devices(id)  
);
```

-- Índices para performance nas consultas de médias

```
CREATE INDEX idx_metrics_device_timestamp ON metrics(device_id,  
timestamp);
```

3. Observações e Desafios no Desenvolvimento

3.1. Centralização da Lógica de Negócio

Um dos principais desafios foi garantir que, independentemente da "porta de entrada" (MQTT, REST ou gRPC), os dados fossem tratados. Para atingir este objetivo implementamos a classe **MetricsProcessor**. Todos os controladores (REST, gRPC Service e MQTT Handler) apenas recebem os dados e encaminham imediatamente para este processador, que centraliza a validação e a gravação.

3.2. Integração do gRPC com Spring Boot

A configuração do gRPC exigiu a definição de ficheiros **.proto** e a geração de código Java (Stubs) durante a compilação.

Para garantir que o servidor gRPC e o servidor web (Tomcat) coexistiam na mesma aplicação Spring Boot sem conflitos de portas configuramos o gRPC para escutar na porta 9090 e o REST na 8080.

3.3. Simulação de Dados Realistas

Para testar o sistema, foi necessário criar clientes que simulassem o comportamento físico da temperatura. Implementou-se uma lógica de "Variação Gradual" (Random Walk) nos clientes, garantindo que a temperatura não saltava de 15°C para 30°C num segundo, mas sim flutuava suavemente, respeitando os limites impostos pelo enunciado.

4. Análise de Performance e Comparação de Protocolos

Com base na implementação e execução do projeto, tivemos de adotar diferenças significativas no comportamento dos três protocolos

Característica	REST (HTTP/1.1)	gRPC (HTTP/2 + Protobuf)	MQTT (TCP Pub/Sub)
Formato de Dados	JSON (Texto - Verboso)	Protobuf (Binário - Compacto)	JSON (no body) + Header Binário
Tamanho do Payload	Grande (chaves repetidas no JSON)	Muito Pequeno (esquema pré-definido)	Médio (depende do body)
Modelo de Comunicação	Pedido-Resposta (Síncrono)	RPC (Síncrono/Assíncrono)	Publish/Subscribe (Assíncrono)
Latência	Média/Alta (Handshake repetitivo)	Baixa (Conexão persistente)	Baixa (Conexão persistente)
Carga no Servidor	Alta (Parsing de texto)	Baixa (Deserialização rápida)	Média (Gestão do Broker)

5. Análise dos Resultados

1. gRPC(Performance):

Foi o protocolo mais eficiente para transferência de dados. O uso de Protobuf eliminou a necessidade de enviar nomes de campos (como "temperature": 22.5) repetidamente, enviando apenas os valores binários mapeados por índices. Observou-se a entrega mais rápida e menor uso de CPU no processamento.

2. MQTT (Eficiência de Rede/IoT):

Embora o corpo da mensagem fosse JSON (por opção de implementação), o protocolo em si demonstrou ser o mais robusto. A capacidade de operar sem bloquear à espera de confirmação do servidor é ideal para cenários de alta latência.

3. REST (Compatibilidade):

Foi o protocolo mais "pesado". Cada envio de métricas exigiu o estabelecimento de headers HTTP completos. No entanto, foi essencial para o **AdminCLI** e para a gestão do sistema, pois permitiu depuração fácil e integração direta com o browser e ferramentas padrão.

5. Conclusão

O sistema híbrido revelou-se a melhor arquitetura: **MQTT** para a "borda" (sensores a bateria), **gRPC** para gateways concentradores de tráfego, e **REST** para a camada de gestão e integração com o utilizador final. Assim conseguimos não comprometer performance por heterogeneidade de protocolos, mas sim melhorar a performance ao utilizar os pontos fortes de cada protocolo para escolher a sua função.